Departament de Ciències de la Computació

Grau en Enginyeria Informàtica – UPC

Facultat d'Informàtica de Barcelona

Pràctica de Cerca Local

Problema de Rescate en Desastres Naturals

Autors:

Eloi Mas Marti
Guillem Revuelta
- - -

Grup de laboratori: G11

Professor: Carlos Fenollosa

2025/2026 – Segon Quadrimestre

27 de març de 2026

Índex

I Plantejament i Anàlisi del Problema **3**

1 Identificació del Problema **4**
1.1 Descripció i Enunciat del Problema 4
1.2 Anàlisi Detallat de les Característiques 5
1.3 Justificació de la Cerca Local 5

II Desenvolupament de la Solució **7**

2 Representació del Problema **8**
2.1 Tractabilitat Computacional 8
2.2 Modelatge de l'Estat i de l'Espai de Cerca 8
2.2.1 Representació dels Estats 8
2.2.2 Índexació d'helicòpters i centres 9
2.2.3 Definició de l'Espai de Cerca 9
2.3 Implementació de la Classe `RescueState` 9
2.3.1 Estructura de dades 9
2.3.2 Constructor còpia 10
2.3.3 Càlcul del cost d'una ruta 10
2.3.4 Càlcul del temps màxim de prioritat 1 10
2.4 Anàlisi de la Mida de l'Espai de Cerca i Eficàcia de la Representació 10
2.4.1 Cota teòrica de l'espai de cerca 10
2.4.2 Eficàcia i eficiència de la representació 10

3 Estat Inicial i Algorisme de Generació **12**
3.1 Generadors de la Solució Inicial 12
3.1.1 Generador Aleatori (`StartRandom`) 12
3.1.2 Generador Golafre (`StartGreedy`) 13
3.1.3 Generador Golafre Millorat (`StartGreedyPro`) 14
3.2 Conclusió 14

4 Operadors de Cerca Local **16**
4.1 Descripció i Funcionalitat dels Operadors 16
4.2 Condicions d'Aplicabilitat i Efectes dels Operadors 16
4.2.1 Operador 1: SwapGroups 16
4.2.2 Operador 2: MoveGroup 17
4.2.3 Operador 3: SwapOrder 17
4.3 Anàlisi del Factor de Ramificació Total 18

4.4 Implementació per a Hill Climbing i Simulated Annealing 18
4.4.1 Hill Climbing (`DesastresSuccessorFunction`) 18
4.4.2 Simulated Annealing (`DesastresSuccessorFunctionSA`) 18
4.5 Justificació de l'Elecció dels Operadors 19
4.6 Conclusió 19

5 Funcions Heurístiques 20
5.1 Introducció i Definició dels Factors 20
5.2 Heurística 1: Temps Total 20
5.3 Heurística 2: Temps Total amb Penalització de Prioritat 21
5.4 Heurística Combinada: $H_1 + w \cdot H_2$ 22
5.5 Conclusió 22

III Experimentació, Resultats i Conclusions 23

6 Descripció d'Algorismes de Cerca Local 24
6.1 Hill Climbing: Principis, Implementació i Pseudocodi 24
6.2 Simulated Annealing: Principis, Implementació i Pseudocodi 24

7 Experiments 26
7.1 Experiment 1: Comparació d'Operadors 26
7.2 Experiment 1.2: Operadors per Escenari 28
7.3 Experiment 2: Estratègia de la Solució Inicial 29
7.4 Experiment 3: Ajust de Paràmetres del Simulated Annealing 31
7.5 Experiment 4: Escalabilitat Proporcional (HC vs SA) 35
7.6 Experiment 5: Escalabilitat Separada (HC) 37
7.7 Experiment 6: Efecte del Nombre d'Helicòpters per Centre 39
7.8 Experiment 7: Heurística H_2 i Ponderació 41

8 Conclusions 44

A Treball en Equip 47
A.1 Distribució de tasques 47

B Treball d'innovació 48
B.1 Ollama 48

Part I

Plantejament i Anàlisi del Problema

Capítol 1

Identificació del Problema

1.1 Descripció i Enunciat del Problema

Per tal de resoldre el problema, primer hem d'establir els elements que hi intervenen. En aquesta pràctica tindrem:

- Una **àrea de rescat** rectangular de 50×50 km, amb l'origen de coordenades a l'extrem inferior esquerre.
- Un nombre G de **grups de persones** distribuïts per l'àrea, cadascun amb coordenades (x_i, y_i), un nombre de persones $n_i \in [1, 12]$ i una prioritat $p_i \in \{1, 2\}$.
- Un nombre C de **centres de rescat** situats als bordes del rectangle, cadascun amb H_c helicòpters.

Els **grups de prioritat 1** contenen ferits i els de **prioritat 2** no. El temps de recollida d'un grup depèn del nombre de persones i de la seva prioritat:

$$t_{\text{recollida}}(g_i) = \begin{cases} 2 \cdot n_i \text{ min} & \text{si } p_i = 1 \\ n_i \text{ min} & \text{si } p_i = 2 \end{cases}$$

Cada **helicòpter** realitza sortides de rescat des del seu centre. En cada sortida recull un o més grups, amb les restriccions següents: màxim **3 grups** per sortida i màxim **15 persones** per sortida. En arribar a les coordenades d'un grup, el recull complet (no es pot dividir un grup). Un cop torna al centre, espera **10 minuts** abans de poder fer una nova sortida. La velocitat de desplaçament és de 100 km/h i la distància entre dos punts es calcula amb la distància euclidiana:

$$d(p_1, p_2) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

El temps de vol entre dos punts és per tant:

$$t_{\text{vol}}(p_1, p_2) = \frac{d(p_1, p_2)}{100} \cdot 60 \text{ min}$$

Donats G grups a rescatar, C centres de rescat i H_c helicòpters per centre, l'objectiu és obtenir, per a cada helicòpter, una llista de sortides de rescat indicant quins grups recull en cada sortida, de manera que es minimitzi un dels criteris de qualitat descrits a la Secció 1.2.

1.2 Anàlisi Detallat de les Característiques

Les solucions han de minimitzar un dels criteris següents:

Criteri 1: Suma de tots els temps emprats pels helicòpters en rescatar tots els grups (sense considerar paral·lelisme):

$$H_1 = \sum_{h=1}^{H_{\text{total}}} \text{costeRuta}(h)$$

Criteri 2: Igual que el Criteri 1, però minimitzant addicionalment el temps que tarda a ser rescatat l'últim grup de prioritat 1, és a dir, el temps des de l'inici del rescat fins que l'últim ferit arriba a un centre:

$$H_2 = H_1 + W \cdot \max_{\substack{g \in \text{grups} \\ p_g = 1}} t_{\text{rescat}}(g)$$

on W és un pes de penalització i $t_{\text{rescat}}(g)$ és el temps total fins que el grup g arriba al centre.

Una solució vàlida ha de complir les restriccions següents:

- Tots els grups han de ser rescatats.
- Cada helicòpter recull els grups complets (no es pot dividir un grup).
- En una sortida, un helicòpter no pot rescatar més de 3 grups.
- En una sortida, un helicòpter no pot transportar més de 15 persones.

1.3 Justificació de la Cerca Local

Quan ens enfrontem a un problema complex, el primer pas és determinar si es pot resoldre de forma exacta en temps raonable. En aquest cas, el nombre de solucions possibles creix de manera factorial amb el nombre de grups. Per a una instància típica de $G = 100$ grups i $H = 5$ helicòpters, el nombre d'assignacions possibles és de l'ordre de $5^{100} \approx 10^{69}$, molt per sobre del que cap algorisme exacte pot explorar.

Una estratègia **golafre** (escollir la millor opció en cada pas) no garanteix una solució òptima global, ja que l'assignació de grups als helicòpters és interdependent: assignar un grup a un helicòpter afecta el cost de les assignacions futures. A més, el problema no presenta subestructura òptima, de manera que tampoc és aplicable la programació dinàmica.

El problema presenta similituds amb el *Vehicle Routing Problem* (VRP), un problema NP-difícil per al qual les millors solucions pràctiques s'obtenen mitjançant metaheurístiques

de cerca local. Donat que no existeix un algorisme eficient per trobar l'òptim global, és justificable renunciar-hi i usar una bona aproximació basada en cerca local.

La cerca local és especialment adequada per a aquest problema pels motius següents:

1. **Solució inicial fàcil:** Qualsevol assignació completa de grups a helicòpters constitueix una solució vàlida. S'han implementat tres estratègies d'inicialització (aleatòria, golafre i golafre millorada) descrites al Capítol 3.
2. **Operadors naturals:** Existeixen transformacions locals clares: intercanviar grups entre helicòpters, moure un grup a un altre helicòpter, o canviar l'ordre de visita dins una ruta. Aquests operadors permeten explorar tot l'espai de solucions (veure Capítol 4).
3. **Funció de qualitat contínua:** La funció heurística és calculable eficientment a partir de l'estat i varia de forma contínua amb les modificacions locals, cosa que permet guiar la cerca cap a solucions millors de forma progressiva.
4. **Absència de condició de parada clara:** No podem saber quan hem assolit l'òptim global, cosa que és característica dels problemes de cerca local on els algorismes s'aturen en un òptim local.

Per aquests motius, hem optat per resoldre el problema mitjançant els algorismes de **Hill Climbing** i **Simulated Annealing**, implementats amb les classes de la llibreria AIMA.

Part II

Desenvolupament de la Solució

Capítol 2

Representació del Problema

2.1 Tractabilitat Computacional

Un aspecte fonamental en el disseny de la solució és l'elecció d'una representació de l'estat que sigui eficient tant en temps com en espai. La cerca local genera una quantitat molt elevada d'estats durant l'exploració —potencialment milions en problemes de mida mitjana— de manera que cada operació sobre l'estat (còpia, modificació, càlcul de la heurística) ha de ser el més ràpida possible.

L'eficiència es mesura principalment en termes de complexitat temporal $O(f(n))$ i complexitat espacial. Per al nostre problema, n representa principalment el nombre de grups G i el nombre total d'helicòpters H. Una representació inadequada podria fer que el càlcul de la heurística fos $O(G^2)$ o pitjor, fent el programa inviable per a instàncies grans. La representació escollida garanteix que totes les operacions crítiques siguin $O(G)$ o millors.

2.2 Modelatge de l'Estat i de l'Espai de Cerca

2.2.1 Representació dels Estats

Un estat del problema representa una assignació completa de tots els grups a helicòpters, incloent l'ordre de visita dins cada ruta. Formalment, definim un estat σ com un parell:

$$\sigma = (\texttt{grupoHeli},\ \texttt{rutas})$$

on:

- $\texttt{grupoHeli} : \{0, \ldots, G-1\} \to \{0, \ldots, H-1\}$ és un vector d'enters que assigna a cada grup g_i l'índex de l'helicòpter h al qual està assignat. Formalment:

  $$\texttt{grupoHeli}[i] = h \iff \text{el grup } i \text{ és assignat a l'helicòpter } h$$

  Si un grup no està assignat (durant la construcció), el valor és -1.

- $\texttt{rutas} : \{0, \ldots, H-1\} \to \text{List}(\mathbb{N})$ és una llista de H llistes d'enters, on $\texttt{rutas}[h]$ conté els índexs dels grups assignats a l'helicòpter h, en l'ordre en què els visitarà.

La relació de consistència entre les dues estructures és:

$$g \in \texttt{rutas}[h] \iff \texttt{grupoHeli}[g] = h$$

Aquesta redundància és intencionada: `grupoHeli` permet consultar en $O(1)$ a quin helicòpter pertany un grup (necessari als operadors), mentre que `rutas` permet iterar eficientment per la ruta de cada helicòpter (necessari al càlcul de la heurística).

2.2.2 Índexació d'helicòpters i centres

Els $H_{\text{total}} = C \cdot H_c$ helicòpters s'indexen de 0 a $H_{\text{total}} - 1$. El centre al qual pertany l'helicòpter h és:

$$\text{centreId}(h) = \left\lfloor \frac{h}{H_c} \right\rfloor$$

on H_c és el nombre d'helicòpters per centre. Aquesta indexació permet accedir al centre corresponent en $O(1)$ sense emmagatzemar explícitament la relació helicòpter-centre.

2.2.3 Definició de l'Espai de Cerca

L'espai de cerca $\mathcal{E}$ és el conjunt de tots els estats σ que compleixen les restriccions del problema:

$$\mathcal{E} = \left\{ \sigma = (\texttt{grupoHeli}, \texttt{rutas}) \;\middle|\; \begin{array}{l} \forall g: \ \texttt{grupoHeli}[g] \in \{0, \ldots, H-1\} \\ \forall h: \ \sum_{g \in \texttt{rutas}[h]} n_g \leq 15 \text{ per sortida} \\ \forall \text{ sortida}: |\text{grups}| \leq 3 \end{array} \right\}$$

Qualsevol estat de $\mathcal{E}$ és una solució vàlida ja que, per construcció, tots els grups estan assignats i les restriccions de capacitat es gestionen al càlcul del cost (les sortides es tanquen automàticament quan es supera la capacitat o el límit de 3 grups).

2.3 Implementació de la Classe `RescueState`

2.3.1 Estructura de dades

La classe `RescueState` implementa la representació descrita amb les dues estructures principals:

- `int[] grupoHeli`: array de mida G, inicialitzat a -1 (cap grup assignat). Ocupa $O(G)$ espai.
- `ArrayList<ArrayList<Integer>> rutas`: llista de H llistes, inicialment buides. Ocupa $O(G + H)$ espai.

Les referències a `Grupos` i `Centros` no es copien entre estats (es comparteixen), ja que són dades estàtiques del problema que mai canvien durant la cerca. Això evita duplicar $O(G + C)$ dades en cada estat nou, reduint significativament el consum de memòria.

2.3.2 Constructor còpia

La cerca local genera una gran quantitat de còpies d'estats (un per cada veí explorat). El constructor còpia té complexitat $O(G + H)$:

- `grupoHeli` es copia amb `clone()` en $O(G)$.
- Cada llista de `rutas` es copia individualment en $O(|\texttt{rutas}[h]|)$, amb un total de $O(G)$ ja que $\sum_h |\texttt{rutas}[h]| = G$.

2.3.3 Càlcul del cost d'una ruta

El mètode `getCosteRuta(h)` recorre la llista `rutas`$[h]$ una sola vegada en $O(|\texttt{rutas}[h]|)$, simulant les sortides successives de l'helicòpter. Per a cada grup, comprova si la sortida actual ha de tancar-se (per excés de persones o de grups), i en cas afirmatiu suma el temps de tornada al centre i els 10 minuts de preparació. El cost total de tots els helicòpters (H_1) es calcula en $O(G)$.

2.3.4 Càlcul del temps màxim de prioritat 1

El mètode `getMaxTiempoP1()` recorre totes les rutes en $O(G)$, mantenint un acumulador de temps per a cada helicòpter. Quan troba un grup de prioritat 1, calcula el temps hipotètic de tornada al centre en aquell moment i actualitza el màxim global. Retorna el temps fins que l'últim ferit és rescatat, utilitzat per la heurística H_2.

2.4 Anàlisi de la Mida de l'Espai de Cerca i Eficàcia de la Representació

2.4.1 Cota teòrica de l'espai de cerca

Sense restriccions, cada grup g_i podria ser assignat a qualsevol dels H helicòpters i col·locat en qualsevol posició de la seva ruta. El nombre total d'estats possibles és de l'ordre de:

$$|\mathcal{E}| \leq H^G \cdot \prod_{h=0}^{H-1} |\texttt{rutas}[h]|!$$

Per a l'escenari base ($G = 100$, $H = 5$), la cota superior és $5^{100} \approx 10^{69}$, sense comptar les permutacions internes de cada ruta. Es tracta d'un espai vastíssim que confirma la necessitat d'una exploració heurística.

2.4.2 Eficàcia i eficiència de la representació

La representació escollida aporta els avantatges següents:

- **Accés $O(1)$ a l'helicòpter d'un grup:** Gràcies a `grupoHeli`, els operadors *SwapGroups* i *MoveGroup* poden localitzar la ruta origen en temps constant, sense haver de cercar en totes les llistes.

- **Modificació local eficient:** Els operadors modifiquen únicament 1 o 2 posicions del vector `grupoHeli` i fan `set`/`remove`/`add` sobre les llistes afectades, tot en $O(G)$ en el pitjor cas (per al *remove* que desplaça elements).
- **Càlcul heurístic $O(G)$:** Tant H_1 com H_2 es calculen en una sola passada per totes les rutes, sense recalcular informació redundant.
- **Memòria $O(G + H)$ per estat:** La mida d'un estat és proporcional al nombre de grups i helicòpters, no al quadrat ni a cap altra funció creixent. Això permet mantenir molts estats en memòria simultàniament durant la cerca.
- **Compartició de dades estàtiques:** Les referències a `Grupos` i `Centros` no es dupliquen entre estats, estalviant memòria de forma significativa quan la cerca genera milers de còpies.

En resum, la representació adopta un equilibri entre redundància controlada (les dues estructures `grupoHeli` i `rutas` contenen informació equivalent) i eficiència operacional, permetent que la cerca local s'executi de forma àgil fins i tot per a instàncies de centenars de grups.

Capítol 3

Estat Inicial i Algorisme de Generació

En aquest capítol s'explica com s'obté la solució inicial que serveix de punt de partida als algorismes de cerca local (Hill Climbing i Simulated Annealing). La qualitat d'aquesta solució inicial és fonamental, ja que afecta directament el nombre d'iteracions necessàries per arribar a un bon òptim local i el cost heurístic final assolit.

Per avaluar l'impacte del punt de partida, s'han implementat tres estratègies de generació:

- Un generador **aleatori** (`StartRandom`), que no té en compte cap criteri de qualitat i produeix una solució inicial de baixa qualitat útil com a referència inferior.
- Un generador **golafre** (`StartGreedy`), que assigna cada grup al centre de rescat més proper, reduint les distàncies de vol inicials.
- Un generador **golafre millorat** (`StartGreedyPro`), que per a cada grup cerca la posició i l'helicòpter que minimitzen l'increment de cost heurístic, obtenint la solució inicial de major qualitat.

3.1 Generadors de la Solució Inicial

3.1.1 Generador Aleatori (`StartRandom`)

Aquest mètode construeix una solució inicial assignant cada grup a un helicòpter escollit uniformement a l'atzar, sense tenir en compte cap criteri de cost ni de distància. Tot i que produeix solucions de baixa qualitat, resulta útil per avaluar la robustesa dels algorismes de cerca local davant d'un punt de partida desfavorable i per contrastar la millora que aporten les estratègies golafre.

Algorithm 1 Generador Aleatori (`StartRandom`)

Require: G = conjunt de grups, H = nombre total d'helicòpters
Ensure: Estat inicial σ amb tots els grups assignats aleatòriament

```
σ ← nou estat buit
for g ← 0 fins a G − 1 do
    h ← enter aleatori uniformement distribuït a [0, H)
    rutas[h].add(g)
    grupoHeli[g] ← h
end for
return σ
```

Complexitat: $O(G)$ en temps i $O(G + H)$ en espai.
Qualitat esperada: Baixa. Els grups s'assignen sense cap criteri geogràfic, generant rutes llargues i desequilibrades entre helicòpters. El valor heurístic H_1 inicial és aproximadament un 70% superior al de `StartGreedyPro` (H inicials $\approx$ 4400 vs. $\approx$ 2700 a l'escenari base).

3.1.2 Generador Golafre (`StartGreedy`)

Aquest mètode implementa una estratègia golafre basada en la **proximitat geogràfica**: per a cada grup, s'assigna al primer helicòpter del centre de rescat més proper. L'objectiu és reduir les distàncies de vol inicials, agrupant cada grup amb el centre que li queda més a prop.

Algorithm 2 Generador Golafre (`StartGreedy`)

Require: G = conjunt de grups, C = conjunt de centres, H_c = helis per centre
Ensure: Estat inicial σ amb assignació per proximitat al centre

```
σ ← nou estat buit
for g ← 0 fins a G − 1 do
    millor_dist ← ∞,   millor_heli ← −1
    for c ← 0 fins a C − 1 do
        dist ← d(coords(g), coords(centre_c))
        if dist < millor_dist then
            millor_dist ← dist
            millor_heli ← c × H_c
        end if
    end for
    rutas[millor_heli].add(g)
    grupoHeli[g] ← millor_heli
end for
return σ
```

Complexitat: $O(G \cdot C)$ en temps.
Qualitat esperada: Mitjana. La proximitat al centre redueix els temps de vol, però l'estratègia no té en compte l'ordre de visita ni la interacció entre grups dins la mateixa ruta. El valor heurístic H_1 inicial és $\approx$ 3100 a l'escenari base.

Limitació: Sempre assigna al *primer* helicòpter del centre més proper ($c \times H_c$), ignorant els helicòpters addicionals del mateix centre quan $H_c > 1$.

3.1.3 Generador Golafre Millorat (`StartGreedyPro`)

Aquest mètode implementa una estratègia golafre d'inserció mínima: per a cada grup, s'exploren **totes les posicions possibles** de **totes les rutes** de tots els helicòpters, i s'escull la combinació (helicòpter, posició) que minimitza l'increment del cost heurístic $\Delta H_1 = \texttt{costeRuta}(h, \sigma \oplus \langle g \text{ a pos} \rangle) - \texttt{costeRuta}(h, \sigma)$.

Aquesta estratègia és anàloga a l'algoritme d'inserció mínima del problema del viatjant de comerç (*cheapest insertion heuristic*), adaptat al context de múltiples vehicles.

Algorithm 3 Generador Golafre Millorat (`StartGreedyPro`)

Require: G = conjunt de grups, H = nombre total d'helicòpters
Ensure: Estat inicial σ de mínima inserció

```
σ ← nou estat buit
for g ← 0 fins a G − 1 do
    millor_inc ← ∞,   millor_heli ← −1,   millor_pos ← −1
    for h ← 0 fins a H − 1 do
        for pos ← 0 fins a |rutas[h]| do
            inc ← calcularIncremento(σ, h, pos, g)
            if inc < millor_inc then
                millor_inc ← inc
                millor_heli ← h,   millor_pos ← pos
            end if
        end for
    end for
    σ.insertarGrupo(millor_heli, millor_pos, g)
end for
return σ
```

La funció `calcularIncremento` insereix temporalment el grup a la posició indicada, calcula el nou cost de la ruta, i el retira immediatament, retornant la diferència de cost. Aquesta operació és $O(|\texttt{rutas}[h]|)$.

Complexitat: $O(G^2 \cdot H)$ en temps, ja que per a cada un dels G grups s'exploren H rutes i fins a $O(G/H)$ posicions per ruta (en promig), cadascuna amb un càlcul $O(G/H)$, resultant en $O(G \cdot H \cdot (G/H) \cdot (G/H)) = O(G^3/H)$ en el pitjor cas.
Qualitat esperada: Alta. En inserir cada grup a la posició de mínim cost, la solució inicial ja s'aproxima a un òptim local. El valor heurístic H_1 inicial és ≈ 2700 a l'escenari base, significativament millor que les alternatives.

3.2 Conclusió

La comparació entre el mètode aleatori i les dues variants golafre posa de manifest la importància d'una bona solució inicial:

- El mètode Random genera un estat inicial sense criteris d'optimització, cosa que sol conduir a un cost global elevat i a una major quantitat d'iteracions durant la cerca.
- Els mètodes Greedy1 i Greedy2 integren en el procés de generació criteris basats en la proximitat i el cost de connexió, oferint solucions inicials de millor qualitat i facilitant la convergència dels algorismes de recerca local.

Capítol 4

Operadors de Cerca Local

4.1 Descripció i Funcionalitat dels Operadors

Un cop definida la representació de l'estat i les estratègies d'inicialització, cal definir els **operadors de transformació** que permeten explorar l'espai de cerca. Cada operador pren un estat vàlid i genera un o més estats veïns modificant localment l'assignació de grups a helicòpters o l'ordre de visita dins una ruta.

S'han implementat tres operadors de transformació, tots ells dissenyats per mantenir la validesa de la solució (tots els grups continuen assignats a exactament un helicòpter):

- **SwapGroups**: intercanvia dos grups entre helicòpters diferents.
- **MoveGroup**: mou un grup d'un helicòpter a un altre, inserint-lo al final de la ruta destí.
- **SwapOrder**: intercanvia dos grups adjacents dins la mateixa ruta d'un helicòpter.

Aquests operadors s'han implementat de forma diferent per a Hill Climbing (classe DesastresSuccessorFunction) i per a Simulated Annealing (classe DesastresSuccessorFunctionSA), tal com s'explica a la Secció 4.4.

4.2 Condicions d'Aplicabilitat i Efectes dels Operadors

4.2.1 Operador 1: SwapGroups

Descripció: Intercanvia dos grups g_1 i g_2 que es troben en helicòpters diferents h_1 i h_2. Després de l'operació, g_1 passa a la posició de g_2 a la ruta de h_2, i g_2 passa a la posició de g_1 a la ruta de h_1.

Condicions d'aplicabilitat:

- $g_1 \neq g_2$
- $\texttt{grupoHeli}[g_1] \neq \texttt{grupoHeli}[g_2]$ (els dos grups han d'estar en helicòpters *diferents*)

Efectes:

- $\texttt{grupoHeli}[g_1] \leftarrow h_2, \quad \texttt{grupoHeli}[g_2] \leftarrow h_1$

- `rutas`$[h_1]$: substitueix g_1 per g_2 a la mateixa posició.
- `rutas`$[h_2]$: substitueix g_2 per g_1 a la mateixa posició.

Factor de ramificació: El nombre de parells (g_1, g_2) amb $g_1 < g_2$ i `grupoHeli`$[g_1] \neq$ `grupoHeli`$[g_2]$ és com a màxim:

$$\binom{G}{2} = \frac{G(G-1)}{2} \approx O(G^2)$$

Per a $G = 100$, això representa fins a 4950 veïns d'aquest operador.

4.2.2 Operador 2: MoveGroup

Descripció: Mou un grup g del seu helicòpter actual h_{orig} a un helicòpter diferent h_{dest}, inserint-lo al final de la ruta de h_{dest}.

Condicions d'aplicabilitat:

- $h_{\text{dest}} \neq h_{\text{orig}}$

Efectes:

- Elimina g de `rutas`$[h_{\text{orig}}]$.
- Afegeix g al final de `rutas`$[h_{\text{dest}}]$.
- `grupoHeli`$[g] \leftarrow h_{\text{dest}}$

Factor de ramificació: Per a cada grup g hi ha $H-1$ helicòpters destí possibles, donant un total de:

$$G \cdot (H-1) \approx O(G \cdot H)$$

Per a $G = 100$ i $H = 5$, això representa 400 veïns d'aquest operador.

4.2.3 Operador 3: SwapOrder

Descripció: Intercanvia dos grups adjacents `rutas`$[h][i]$ i `rutas`$[h][i+1]$ dins la ruta d'un mateix helicòpter h. Aquest operador permet optimitzar l'ordre de visita sense canviar l'assignació de grups a helicòpters.

Condicions d'aplicabilitat:

- $|$`rutas`$[h]| \geq 2$ (la ruta ha de tenir almenys dos grups)

Efectes:

- Intercanvia `rutas`$[h][i]$ i `rutas`$[h][i+1]$.
- `grupoHeli` no canvia (els grups continuen al mateix helicòpter).

Factor de ramificació: Per a cada helicòpter h amb k_h grups, hi ha $k_h - 1$ intercanvis adjacents possibles. En total:

$$\sum_{h=0}^{H-1} (|\texttt{rutas}[h]| - 1) \leq G - H \approx O(G)$$

Per a $G = 100$ i $H = 5$, fins a 95 veïns d'aquest operador.

4.3 Anàlisi del Factor de Ramificació Total

El factor de ramificació total del conjunt d'operadors per a Hill Climbing és la suma dels tres:

$$F_{\text{HC}} = \frac{G(G-1)}{2} + G(H-1) + (G-H) = \frac{G(G-1)}{2} + GH - G - H \approx O(G^2)$$

Per a l'escenari base ($G = 100$, $H = 5$):

Taula 4.1: Factor de ramificació per operador (escenari base: $G = 100$, $H = 5$)

Operador	**Fórmula**	**Valor ($G = 100$, $H = 5$)**
SwapGroups	$G(G-1)/2$	4950
MoveGroup	$G(H-1)$	400
SwapOrder	$G-H$	95
Total		**5445**

El factor de ramificació és dominat per **SwapGroups** ($\approx 91\%$ del total), que és quadràtic en G. Això explica per què el temps d'execució de HC creix de forma superlineal quan augmenta el nombre de grups, tal com s'observa a l'Experiment 5 (Secció **??**).

4.4 Implementació per a Hill Climbing i Simulated Annealing

La implementació dels operadors és diferent per als dos algorismes, seguint el requisit de la pràctica:

4.4.1 Hill Climbing (`DesastresSuccessorFunction`)

Per a HC, el mètode `getSuccessors` genera **tots** els veïns possibles aplicant els tres operadors de forma exhaustiva. HC necessita conèixer tots els successors per poder escollir el millor, de manera que es retorna una llista completa de fins a 5445 estats (per a l'escenari base).

El cost per iteració de HC és $O(G^2)$ per generar els successors més $O(G)$ per calcular la heurística de cada un, donant un cost total per pas de $O(G^3)$ en el pitjor cas.

4.4.2 Simulated Annealing (`DesastresSuccessorFunctionSA`)

Per a SA, el mètode `getSuccessors` genera **un sol veí** aleatori per iteració. L'algorisme escull aleatòriament un dels tres operadors (amb probabilitat 1/3 cadascun) i aplica paràmetres també escollits a l'atzar:

- **SwapGroups**: escull g_1 i g_2 aleatoris (amb fins a 50 intents per trobar un parell vàlid en helicòpters diferents).

- **MoveGroup**: escull un grup g i un helicòpter destí h_{dest} aleatoris, i insereix g en una posició aleatòria de la ruta destí.
- **SwapOrder**: escull un helicòpter amb almenys 2 grups i intercanvia dos grups adjacents en una posició aleatòria.

Si no es pot generar cap veí vàlid (cas molt poc freqüent), es retorna el mateix estat sense modificació ($NoOp$).

El cost per iteració de SA és $O(G)$ per generar l'únic successor, cosa que permet executar moltes més iteracions en el mateix temps respecte a HC.

4.5 Justificació de l'Elecció dels Operadors

La selecció d'aquests tres operadors es justifica per dos criteris principals:

1. **Connectivitat de l'espai de cerca:** La combinació de SwapGroups, MoveGroup i SwapOrder permet assolir qualsevol solució vàlida des de qualsevol altra en un nombre finit de passos. SwapGroups i MoveGroup permeten redistribuir grups entre helicòpters, mentre que SwapOrder optimitza l'ordre de visita dins cada ruta. Sense SwapOrder, dues solucions que diferissin únicament en l'ordre de visita d'un helicòpter estarien desconnectades.
2. **Equilibri entre ramificació i qualitat:** Un factor de ramificació massa petit limita l'exploració i pot deixar l'algorisme atrapat en òptims locals pobres. Un factor massa gran augmenta el temps per iteració.

4.6 Conclusió

Els tres operadors implementats formen un conjunt complet i eficient per explorar l'espai de solucions del problema de rescat. SwapGroups domina el factor de ramificació i permet reconfigurar l'assignació global de grups, MoveGroup afina la distribució entre helicòpters de forma gradual, i SwapOrder optimitza l'ordre de visita sense modificar l'assignació. La seva implementació diferenciada per a HC i SA permet adaptar el comportament de l'exploració a les necessitats de cada algorisme.

Capítol 5

Funcions Heurístiques

5.1 Introducció i Definició dels Factors

La funció heurística guia l'exploració de l'espai de cerca cap a solucions de major qualitat. En el nostre problema, l'objectiu és minimitzar el temps total de rescat, amb la possibilitat d'introduir un segon criteri que penalitzi el retard en l'atenció als ferits. Definim per a una solució σ les dues magnituds bàsiques:

- $f(\sigma)$: **Temps total de rescat**, és a dir, la suma del temps emprat per cada helicòpter en rescatar tots els grups que té assignats:

  $$f(\sigma) = \sum_{h=0}^{H-1} \texttt{costeRuta}(h)$$

  on $\texttt{costeRuta}(h)$ inclou tots els temps de vol, de recollida i d'espera entre sortides de l'helicòpter h.

- $p(\sigma)$: **Temps màxim fins al darrer ferit rescatat**, és a dir, el temps des de l'inici fins que l'últim grup de prioritat 1 arriba a un centre de rescat:

  $$p(\sigma) = \max_{\substack{g \in \text{grups} \\ \text{prioritat}(g)=1}} t_{\text{rescat}}(g)$$

A partir d'aquestes dues magnituds es defineixen les dues funcions heurístiques implementades, totes dues de minimització.

5.2 Heurística 1: Temps Total

$$H_1(\sigma) = f(\sigma) = \sum_{h=0}^{H-1} \texttt{costeRuta}(h)$$

Descripció: Retorna directament la suma dels costos de totes les rutes de tots els helicòpters. Cada cost de ruta inclou: els temps de vol entre punts consecutius de la ruta, els temps de recollida de cada grup (1 minut per persona si prioritat 2, 2 minuts per persona si prioritat 1), i els 10 minuts d'espera entre sortides consecutives del mateix helicòpter.

Justificació: Aquesta funció mesura l'eficiència global del sistema de rescat sense distingir entre grups ferits i sans. Minimitzar H_1 equival a minimitzar el treball total dels helicòpters, afavorint rutes curtes i ben equilibrades entre helicòpters.

Propietats:

- És **consistent**: qualsevol operació que redueixi el temps total d'una o més rutes reduirà H_1.
- **No distingeix prioritats**: tracta un grup de ferits exactament igual que un grup de sans (excepte pel temps de recollida, que ja és el doble). Això pot conduir a solucions on alguns ferits esperen molt temps si el seu rescat no és rendible globalment.
- **Complexitat de càlcul:** $O(G)$, ja que recorre totes les rutes una sola vegada.

5.3 Heurística 2: Temps Total amb Penalització de Prioritat

$$H_2(\sigma) = f(\sigma) + W \cdot p(\sigma) = \sum_{h=0}^{H-1} \texttt{costeRuta}(h) + W \cdot \max_{\substack{g \\ p_g=1}} t_{\text{rescat}}(g)$$

on $W = 10$ és el pes de penalització fixat per la implementació.

Descripció: Afegeix a H_1 una penalització proporcional al temps que tarda a ser rescatat l'últim grup ferit. Quan l'algorisme minimitza H_2, té un incentiu explícit per atendre aviat els grups de prioritat 1, fins i tot si això suposa un cert empitjorament de l'eficiència global.

Justificació: En un context real de desastre, l'atenció urgent als ferits és una prioritat mèdica i ètica. H_2 formalitza aquest criteri introduint un cost addicional proporcional al retard màxim en l'atenció als ferits. El pes $W = 10$ fa que el temps d'espera del darrer ferit tingui un impacte equivalent a 10 vegades el seu valor en minuts sobre la funció heurística.

Diferència fonamental respecte a H_1: H_1 optimitza l'eficiència global tractant tots els grups de la mateixa manera. H_2 introdueix una *asimetria de prioritat*: el terme $W \cdot p(\sigma)$ penalitza especialment les solucions en què un ferit queda per rescatar al final, independentment del cost global. Dos solucions amb el mateix $f(\sigma)$ però diferent $p(\sigma)$ seran ordenades diferentment per H_2.

Propietats:

- **Dominància:** $H_2(\sigma) \geq H_1(\sigma)$ per a tot σ, ja que $W \cdot p(\sigma) \geq 0$.
- **Sensibilitat al pitjor cas:** El terme max fa que H_2 sigui sensible al *pitjor* ferit, no a la mitjana. Això motiva l'algorisme a distribuir els ferits de forma equilibrada entre helicòpters.
- **Complexitat de càlcul:** $O(G)$, igual que H_1.

5.4 Heurística Combinada: $H_1 + w \cdot H_2$

Per a l'Experiment 7, s'utilitza una funció combinada paramètrica:

$$H_{\text{comb}}(\sigma, w) = H_1(\sigma) + w \cdot H_2(\sigma) = (1 + w) \cdot f(\sigma) + w \cdot W \cdot p(\sigma)$$

amb $w \in \{0, 1, 2, 4, 8, 16\}$. El cas $w = 0$ correspon a H_1 pura (sense penalització de prioritat).

Justificació de la ponderació: El paràmetre w permet controlar el *trade-off* entre eficiència global i urgència dels ferits. Valors petits de w prioritzen H_1 (eficiència global), mentre que valors grans prioritzen $p(\sigma)$ (atenció ràpida als ferits). Doblant w progressivament s'observa com canvia l'equilibri entre H_1 i H_2 a la solució final.

5.5 Conclusió

Les dues funcions heurístiques implementades cobreixen els dos criteris de qualitat de l'enunciat. H_1 és la funció d'eficiència global, adequada quan tots els grups es tracten de la mateixa manera. H_2 afegeix una penalització per retard en l'atenció als ferits, introduint una asimetria de prioritat que és rellevant en contextos reals de rescat. La funció combinada $H_1 + w \cdot H_2$ permet explorar el *trade-off* entre ambdós criteris de forma sistemàtica mitjançant el paràmetre w.

Part III

Experimentació, Resultats i Conclusions

Capítol 6

Descripció d'Algorismes de Cerca Local

En aquest capítol es presenten els dos algorismes de cerca local emprats en aquesta pràctica d'Intel·ligència Artificial: Hill Climbing i Simulated Annealing. Tant en la seva formulació teòrica com en la seva implementació, es treballa amb un espai de solucions $\mathcal{S}$ i una funció heurística $h : \mathcal{S} \to \mathbb{R}$. Per tal de convertir aquesta funció en una mesura de qualitat que s'ha de maximitzar, es defineix:

$$\text{valor}(s) = -1 \cdot h(s),$$

de manera que una solució amb menor heurística (i, per tant, millor) té un valor més alt

6.1 Hill Climbing: Principis, Implementació i Pseudocodi

L'algorisme de Hill Climbing inicia amb un node inicial s_0 i explora iterativament l'espai de solucions. En cada iteració, s'expandeix el node actual per obtenir els seus veïns (o fills) i es selecciona aquell node que posseeix el màxim valor—és a dir, el que té la menor heurística, ja que $\text{valor}(s) = -h(s)$. Si cap dels nodes veïns presenta una millora respecte al node actual, s'ha arribat a un òptim local i l'algorisme finalitza.

Pseudocodi

Algorithm 4 Hill Climbing (Implementació)

```
s ← s_0                                          ▷ Inicialització amb la solució inicial
while existeix un node s' entre els fills de s amb valor(s') > valor(s) do
    s ← s'                                       ▷ Actualitzar amb el node amb el màxim valor
end whilereturn s                                ▷ Retorna el node òptim local
```

6.2 Simulated Annealing: Principis, Implementació i Pseudocodi

L'algorisme de Simulated Annealing té com a objectiu evitar l'atrapament en òptims locals permetent, de manera controlada, l'acceptació de solucions pitjors. Inspirat en el

procés de refredament dels metalls, aquest algorisme utilitza un *scheduler* per determinar la temperatura T a cada iteració. Els paràmetres inicials són:

- $s_0 \in \mathcal{S}$: solució inicial.
- $T_0 > 0$: temperatura inicial.
- Un *scheduler* que redueix la temperatura segons una regla de refredament.

El procediment és el següent:

1. **Inicialització:** S'estableix el node actual $s \leftarrow s_0$, s'inicialitza la temperatura amb $T \leftarrow T_0$ i es guarda el millor node trobat, $\mathit{best} \leftarrow s$.
2. **Iteració:** Per a cada pas:
   (a) Es calcula la temperatura actual mitjançant el *scheduler*: $T = \mathit{scheduler}$ `.getTemp`(step). Si $T = 0$, es finalitza la cerca.
   (b) Es selecciona aleatòriament un node s' entre els fills (veïns) del node actual.
   (c) Es calcula la diferència de valor:
   $$\Delta E = \text{valor}(s') - \text{valor}(s).$$
   (d) Es genera un nombre aleatori $\mathit{al} \in [0, 1]$ i es calcula:
   $$\text{prob} = \frac{1}{1 + \exp\left(\dfrac{\Delta E}{T}\right)}.$$
   (e) **Acceptació:**
   - Si $\Delta E > 0$, es considera que s' és una solució millor i s'accepta automàticament.
   - Si $\Delta E \leq 0$, s'accepta s' amb probabilitat $1 - \text{prob}$, és a dir, si $\mathit{al} > \text{prob}$.

   (f) S'actualitza el millor node si es troba que $\text{valor}(s') > \text{valor}(\mathit{best})$.
3. **Terminació:** El procés es repeteix fins a completar el nombre de passos predefinit o fins que la temperatura arribi a zero. Es retorna el millor node trobat.

Capítol 7

Experiments

Tots els experiments s'han executat amb 100 rèpliques (semilles fixes) per garantir la reproduïbilitat i obtenir estimacions estadísticament robustes. L'escenari base és: 5 centres, 1 helicòpter per centre, 100 grups. La mètrica principal és H_1 (temps total de rescat de tots els grups); en l'Experiment 7 s'afegeix H_2 (temps màxim fins al darrer ferit). En cada secció es documenten la hipòtesi, els resultats numèrics i les conclusions que motiven les decisions de disseny preses.

7.1 Experiment 1: Comparació d'Operadors

Observació	El conjunt d'operadors de veïnat determina l'espai que HC pot explorar i, per tant, la qualitat de la solució final.
Plantejament	Compararem les set combinacions possibles dels tres operadors bàsics: *SwapGroups* (SG), *MoveGroup* (MG) i *SwapOrder* (SO).
Hipòtesi	SG serà el motor principal de millora perquè reorganitza l'assignació de grups entre helicòpters. Afegir-hi MG aportarà una millora addicional. SO sol serà el menys efectiu, ja que no modifica les assignacions, només l'ordre de visita.
Mètode	Escenari base. HC, H_1, inicialització Random. 100 rèpliques per operador. Es mesuren H_{inicial}, H_{final}, temps i nodes expandits.

Taula 7.1: Plantejament de l'Experiment 1

Resultats

Operadors	H_{inicial}	H_{final}	Reducció (%)	Temps (ms)
SG	4 418.8	2 660.7	39.8%	309.7
MG	4 386.1	2 879.8	34.3%	104.9
SO	4 420.6	4 064.9	8.0%	3.8
SG + MG	4 386.8	2 609.9	40.5%	681.5
SG + SO	4 407.5	2 656.6	39.7%	295.9
MG + SO	4 401.7	2 880.8	34.6%	59.5
SG + MG + SO	4 408.1	2 608.5	40.8%	383.8

Taula 7.2: Comparació d'operadors — mitjanes de 100 rèpliques

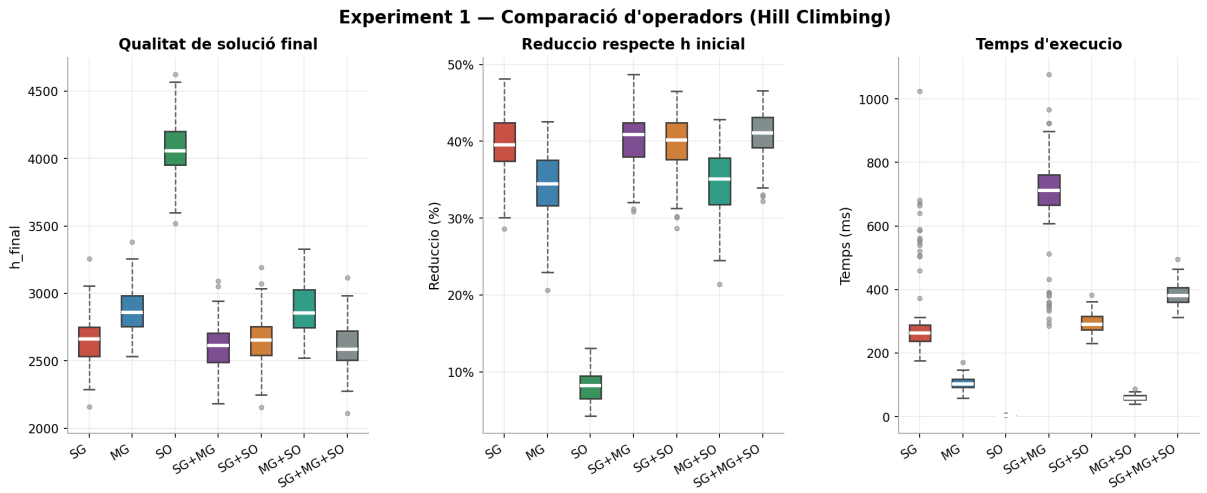

Figura 7.1: Boxplots de H_{final}, reducció relativa (%) i temps d'execució per a cada combinació d'operadors (100 rèpliques). La caixa mostra el rang interquartílic, la línia blanca la mediana i els punts els outliers.

Anàlisi i conclusions

Els resultats confirmen plenament la hipòtesi. *SwapOrder* en solitari és clarament el pitjor operador, amb una reducció de tan sols un 8% sobre H_{inicial}: reordenar la seqüència de visita sense modificar les assignacions deixa el HC en un mínim local molt superficial. Qualsevol combinació que inclogui SG obté reduccions al voltant del 40%, cosa que confirma que l'operador de reassignació de grups és el motor principal de millora.

La combinació SG+MG+SO obté la millor H_{final} de mitjana (2 608.5), seguida molt de prop per SG+MG (2 609.9). La diferència és marginal ($< 0.1\%$), però la combinació completa té un avantatge en diversitat de moviments que pot resultar rellevant en instàncies difícils. El cost addicional en temps de SG+MG+SO respecte a SG+SO (383.8 ms vs. 295.9 ms) es justifica per la lleugera millora en qualitat.

Cal destacar que les combinacions que no inclouen SG (MG, SO i MG+SO) mai superen el 35% de reducció, malgrat tenir un cost temporal molt inferior. Això suggereix que l'espai de cerca accessible sense SG és fonamentalment limitat.

Decisió: Fixem `SwapGroups+MoveGroup+SwapOrder` (`DesastresSuccessorFunction`) com a operador de referència per a la resta d'experiments.

7.2 Experiment 1.2: Operadors per Escenari

Observació	La tria d'operadors pot interactuar amb el nombre de centres i helicòpters.
Plantejament	Comparem SG, SG+SO i SG+MG+SO en dos escenaris (5 i 10 centres) amb 1, 2 i 3 helicòpters per centre.
Hipòtesi	L'avantatge de SG+MG+SO sobre SG serà més pronunciat quan hi hagi més helicòpters, ja que l'operador MG guanya rellevància quan hi ha moltes opcions de reassignació.
Mètode	HC, H_1, GreedyPro. 100 rèpliques per combinació.

Taula 7.3: Plantejament de l'Experiment 1.2

Resultats

Centres	Helis/c	SG	SG+SO	SG+MG+SO
5	1	2 758.6	2 742.1	2 700.0
	2	2 665.6	2 657.9	2 609.6
	3	2 611.2	2 604.2	2 548.5
10	1	2 503.9	2 497.1	2 426.9
	2	2 394.4	2 384.9	2 304.3
	3	2 308.8	2 304.2	2 193.4

Taula 7.4: H_{final} mitjana per escenari i operadors — 100 rèpliques

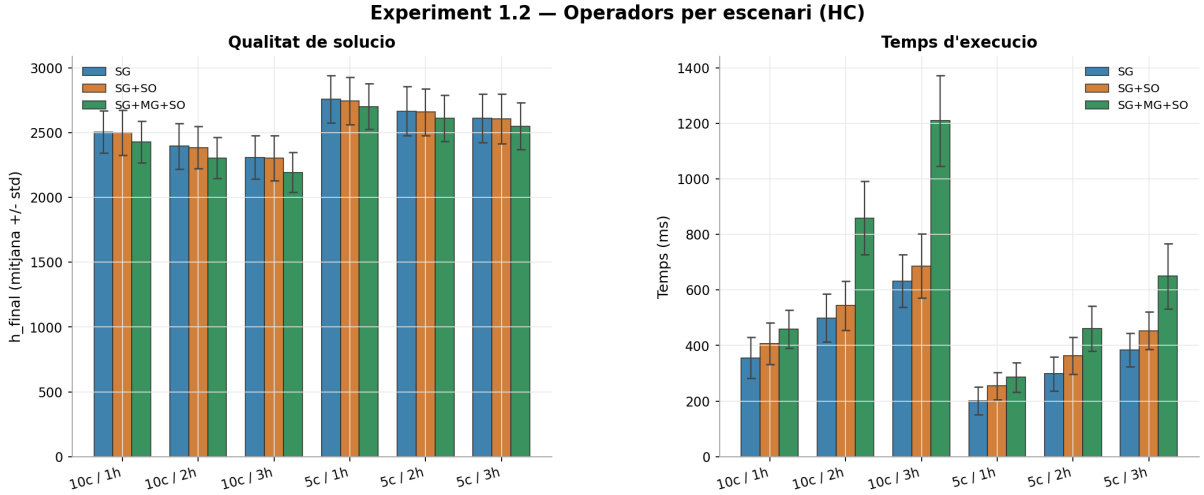

Figura 7.2: H_{final} i temps d'execució per a cada combinació d'operadors i escenari (centres/helicòpters). Barres agrupades per escenari; les barres d'error indiquen ±1 desviació estàndard.

Anàlisi i conclusions

L'avantatge de SG+MG+SO sobre SG s'intensifica clarament a mesura que augmenten els helicòpters: amb 5 centres i 1 heli/centre la diferència és de 58.6 punts, i amb 10 centres i 3 helis/centre arriba als 115.4 punts. Això confirma la hipòtesi: l'operador MG és especialment útil quan hi ha múltiples helicòpters per centre, ja que permet moure un grup d'un helicòpter a un altre del mateix centre, un moviment que SG no pot realitzar.

La combinació SG+SO obté resultats intermedis en tots els escenaris, sempre entre SG i SG+MG+SO, cosa que indica que SO aporta un benefici modest però consistent en combinar-se amb SG.

7.3 Experiment 2: Estratègia de la Solució Inicial

Observació	L'estratègia d'inicialització determina el punt de partida de HC i pot condicionar tant la qualitat final com el temps de convergència.
Plantejament	Comparem tres generadors: *Random* (assignació aleatòria), *Greedy* (assignació voraç simple) i *GreedyPro* (greedy amb optimització local de l'ordre de ruta).
Hipòtesi	Partir d'una millor solució inicial portarà a una millor solució final i en menys temps, ja que el camí fins a l'òptim local és més curt. GreedyPro serà la millor estratègia en qualitat i temps.
Mètode	Escenari base. HC, H_1, SG+MG+SO. 100 rèpliques per estratègia.

Taula 7.5: Plantejament de l'Experiment 2

Resultats

Estratègia	H_{inicial}	H_{final} (mitjana ± std)	Millora HC	Temps (ms)
Random	4 431.2	2 739.7 ± 189.7	38.2%	201.4
Greedy	3 018.1	2 701.7 ± 178.0	10.5%	88.8
GreedyPro	2 667.0	2 603.0 ± 174.1	2.4%	38.6

Taula 7.6: Comparació d'estratègies inicials — mijanes de 100 rèpliques

Els t-tests bilaterals entre estratègies mostren diferències estadísticament significatives entre GreedyPro i les altres dues (Random vs. GreedyPro: $p < 0.0001$; Greedy vs. GreedyPro: $p = 0.0001$), però no entre Random i Greedy ($p = 0.146$).

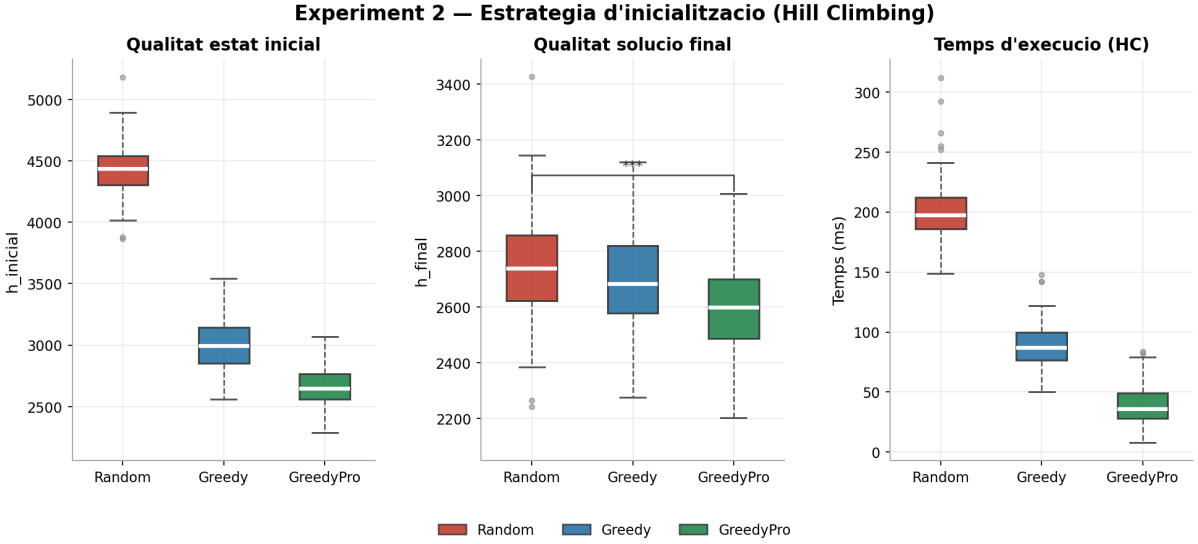

Figura 7.3: Boxplots de H_{inicial}, H_{final} i temps d'execució per a cada estratègia d'inicialització (100 rèpliques). El bracket sobre la caixa de H_{final} indica la significació estadística del t-test entre Random i GreedyPro (*** $p < 0.001$).

Anàlisi i conclusions

GreedyPro és superior en tots els criteris: obté la millor H_{final} (2 603.0), la millor solució inicial i el menor temps total (38.6 ms, gairebé 5× menys que Random). La baixa millora relativa del HC sobre GreedyPro (2.4% enfront del 38.2% de Random) indica que GreedyPro ja entrega una solució molt propera a un òptim local, reduint el treball de HC a un affinament mínim.

El resultat sorprenent és que Random i Greedy no difereixen significativament en qualitat final ($p = 0.146$), tot i que Greedy part d'una H_{inicial} molt millor. Això suggereix que el paisatge de cerca des d'una solució aleatòria conté camins de descens igualment bons, però que HC triga molt més a recórrer-los (201 ms vs. 89 ms).

Decisió: Fixem `GreedyPro` com a estratègia d'inicialització per a la resta d'experiments.

7.4 Experiment 3: Ajust de Paràmetres del Simulated Annealing

Observació	Els paràmetres k, λ, *steps* i *stiter* del SA influeixen en com s'explora l'espai de solucions i, per tant, en la qualitat final.
Plantejament	Explorem l'efecte de cada paràmetre per separat en tres sub-experiments: (3.1) variació de *steps*, (3.2) graella $k \times \lambda$, (3.3) variació de *stiter*.
Hipòtesi	Augmentar *steps* millorarà la solució fins a un punt de saturació. λ petit (refredament lent) explorarà millor l'espai. k gran permetrà acceptar pitjors transicions a temperatures iguals, facilitant l'escapament de mínims locals. *stiter* petit serà millor perquè amb *steps* fix implica més nivells de temperatura.
Mètode	Escenari base, H_1, SG+MG+SO (versió SA), Greedy-Pro. 10 rèpliques per configuració.

Taula 7.7: Plantejament de l'Experiment 3

3.1: Efecte del nombre de passos (*steps*)

steps	H_{final}	**Reducció sobre HC**
5 000	2 670.5	−2.0%
10 000	2 649.1	−1.7%
20 000	2 637.3	−1.3%
50 000	2 607.8	+0.2%
75 000	2 599.6	+0.5%
100 000	2 596.6	+0.6%
150 000	2 605.3	+0.3%
200 000	2 603.1	+0.4%
HC (referència)		2 603.0

Taula 7.8: Efecte de *steps* sobre H_{final} — $k = 10$, $\lambda = 0.001$, *stiter*=100

El SA supera el HC a partir de *steps* $\approx$ 50 000. La millora es satura ràpidament: entre 100 000 i 200 000 steps la H_{final} pràcticament no varia (2 596.6 vs. 2 603.1). Fixem *steps* = 150 000 com a valor de referència, equilibrant qualitat i temps d'execució.

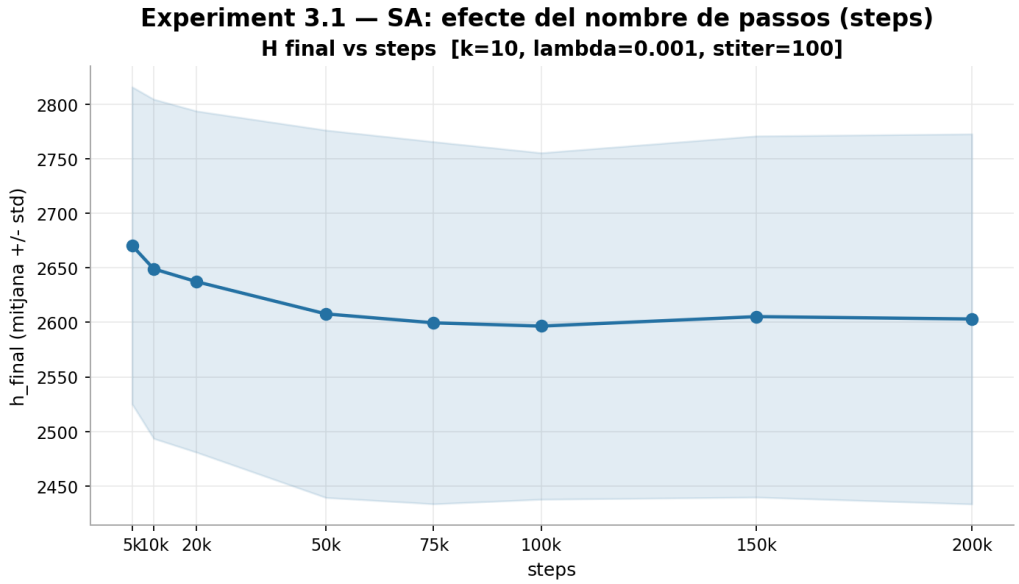

Figura 7.4: H_{final} del SA en funció del nombre de passos (*steps*). La banda ombrejada indica ±1 desviació estàndard sobre 10 rèpliques. La línia horitzontal puntejada marcaria la referència de HC.

3.2: Efecte de $k \times \lambda$

k	λ	$H_{\textbf{final}}$
10	0.001000	2 590.6
100	0.000100	2 593.8
10	0.000800	2 595.7
50	0.000100	2 596.3
500	0.000100	2 597.7
10	0.000100	2 602.0
HC (referència)		2 603.0

Taula 7.9: Top 6 combinacions $k \times \lambda$ — *steps*=150 000, *stiter*=100, 10 rèpliques

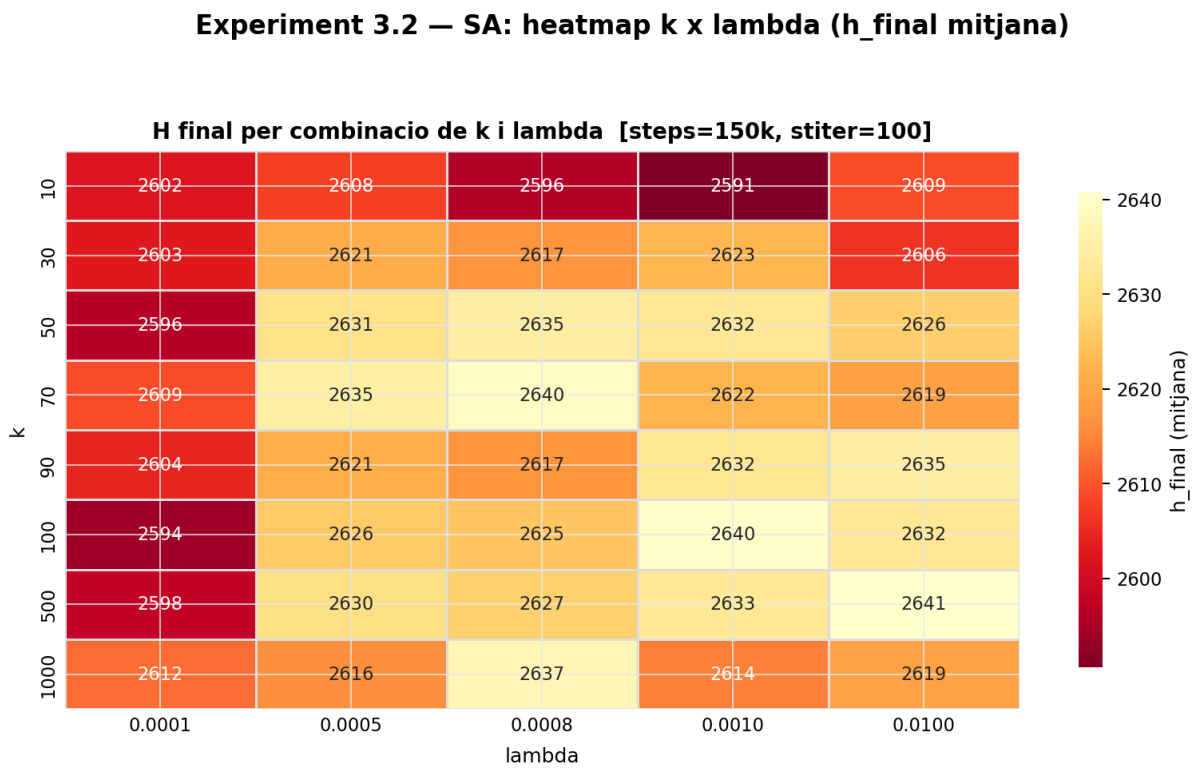

Figura 7.5: Mapa de calor de H_{final} mitjana per a cada combinació $k \times \lambda$ (*steps*=150 000, *stiter*=100). Els valors més baixos (millors) apareixen en colors més freds.

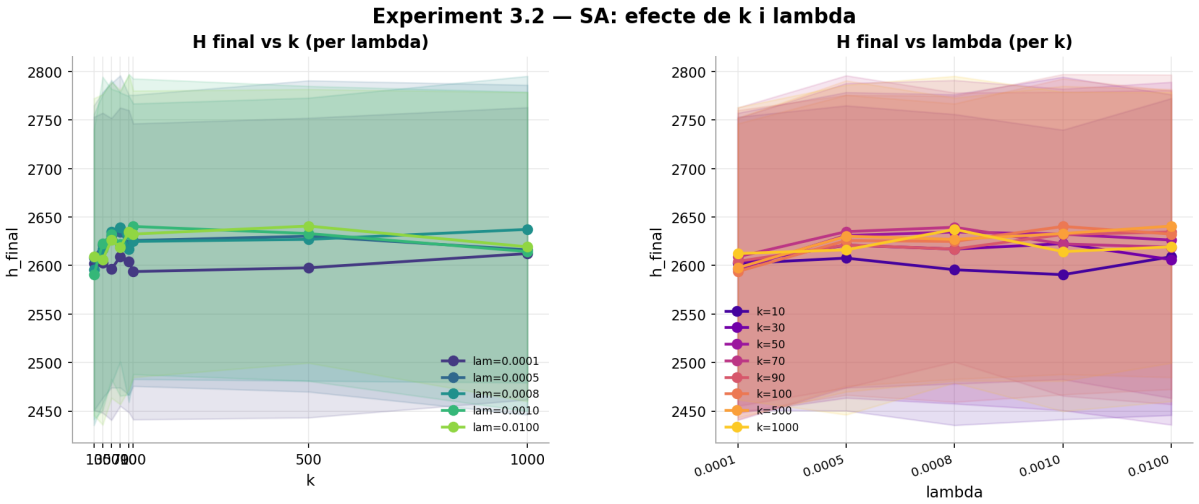

Figura 7.6: H_{final} en funció de k (esquerra) i de λ (dreta), desglossada per l'altre paràmetre. La banda ombrejada indica ± 1 desviació estàndard.

La millor combinació és $k = 10$, $\lambda = 0.001$, que obté $H_{\text{final}} = 2\,590.6$, un 0.5% millor que HC. Els valors λ molt petits (< 0.0001) tendeixen a empitjorar la solució perquè el refredament és massa lent i el SA no convergeix en les iteracions disponibles. Valors de k molt grans (> 500) tampoc milloren, indicant que la probabilitat d'acceptar transicions dolentes ja és prou gran amb $k = 10$.

3.3: Efecte de *stiter*

stiter	H_{final} (mitjana ± std)
10	$2\,583.9 \pm 176.9$
100	$2\,586.2 \pm 172.4$
500	$2\,588.3 \pm 176.6$
1 000	$2\,595.3 \pm 171.1$
5 000	$2\,608.4 \pm 178.9$
10 000	$2\,611.7 \pm 175.8$
50 000	$2\,615.2 \pm 176.8$

Taula 7.10: Efecte de *stiter* — $k = 10$, $\lambda = 0.001$, *steps*=150 000, 100 rèpliques

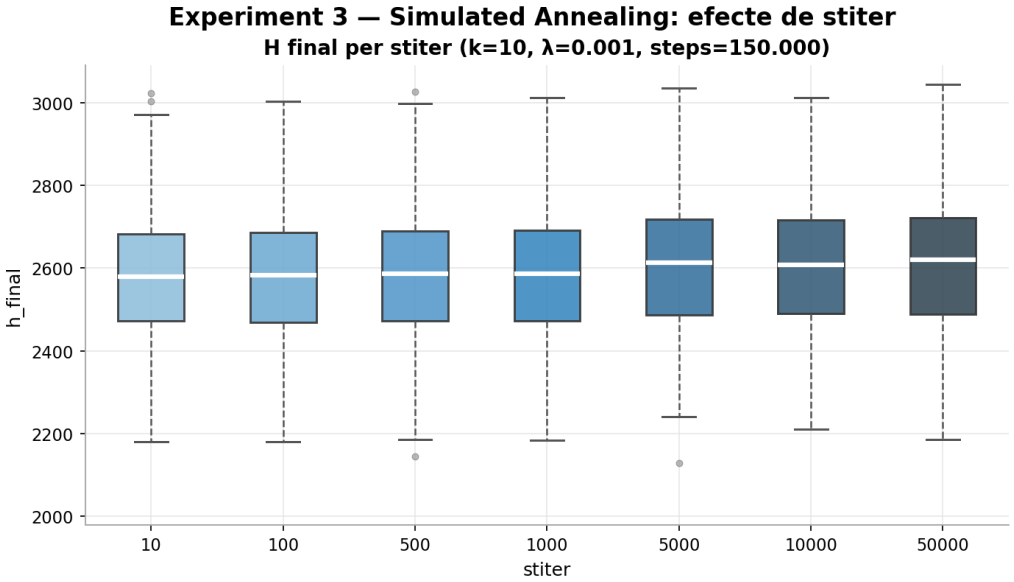

Figura 7.7: Boxplots de H_{final} per a cada valor de *stiter* ($k = 10$, $\lambda = 0.001$, *steps*=150 000, 100 rèpliques). La degradació de blau indica valors creixents de *stiter*.

Contràriament al que podria semblar, *stiter* petit obté millors resultats. Amb *steps* fixat, *stiter* gran implica menys nivells de temperatura, de manera que la temperatura baixa massa ràpidament i el SA queda encallat prematurament. *stiter* = 10 dóna la millor H_{final} (2 583.9), però les diferències entre els valors petits (≤ 100) no són estadísticament rellevants donada l'alta desviació estàndard (≈ 175).

Decisió: Fixem $k = 10$, $\lambda = 0.001$, *steps* = 150 000, *stiter* = 100 per als experiments posteriors.

7.5 Experiment 4: Escalabilitat Proporcional (HC vs SA)

Observació	El temps d'execució i la qualitat relativa dels dos algorismes pot variar en funció del tamany del problema.
Plantejament	Augmentem els paràmetres seguint la proporció 5 centres : 100 grups, fins a 25c/500g.
Hipòtesi	El temps de HC creixerà superlinealment. SA mantindrà un temps aproximadament constant gràcies al nombre fix de *steps*, però perdrà qualitat relativa amb problemes grans perquè els mateixos *steps* han d'explorar un espai molt més gran.
Mètode	5 escenaris: 5c/100g, 10c/200g, 15c/300g, 20c/400g, 25c/500g. HC i SA, H_1, GreedyPro. 10 rèpliques per tamany.

Taula 7.11: Plantejament de l'Experiment 4

Resultats

n_c	n_g	H_{HC}	T_{HC} (ms)	H_{SA}	T_{SA} (ms)	ΔH (%)
5	100	2 617.5	48.1	2 600.9	192.3	−0.6%
10	200	4 632.9	768.2	4 641.3	334.3	+0.2%
15	300	6 577.5	5 293.7	6 659.6	490.1	+1.2%
20	400	8 443.2	18 214.4	8 589.0	677.3	+1.7%
25	500	10 349.7	59 393.0	10 590.6	869.8	+2.3%

Taula 7.12: Escalabilitat proporcional — HC vs. SA, 10 rèpliques. ΔH = diferència relativa SA respecte HC.

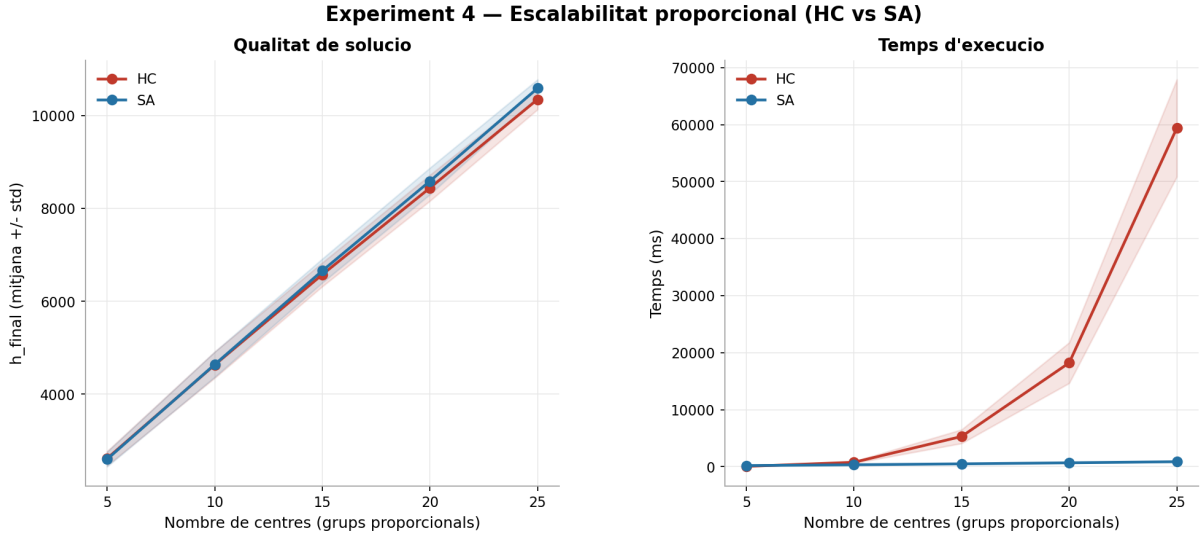

Figura 7.8: H_{final} (esquerra) i temps d'execució (dreta) en funció del nombre de centres, amb grups proporcionals (5c:100g). La banda ombrejada indica ± 1 desviació estàndard sobre 10 rèpliques.

Anàlisi i conclusions

El temps de HC creix de forma molt superlineal: passar de 5c/100g a 25c/500g (factor 5× en tamany) implica un increment de temps d'aproximadament 1 235× (de 48 ms a 59 393 ms), consistent amb la complexitat dominant $O(G^2)$ per iteració del operador SwapGroups.

El SA manté un temps quasi constant (192–870 ms), incrementant de forma gairebé lineal degut al creixement del cost per iteració. No obstant, la seva qualitat relativa empitjora progressivament: a l'escenari base SA és un 0.6% millor que HC, però a 25c/500g és un 2.3% pitjor. Això confirma que amb *steps* fix, el SA explora una fracció cada vegada menor de l'espai de cerca a mesura que el problema creix.

Per a instàncies grans (a partir de ~ 15 centres) HC és millor en qualitat tot i ser molt més lent. SA seria preferible si el temps d'execució fos un factor crític, però cal augmentar *steps* proporcionalment al tamany per mantenir la qualitat.

7.6 Experiment 5: Escalabilitat Separada (HC)

Observació	Augmentar grups i augmentar centres afecten el problema de maneres molt diferents: els grups defineixen la càrrega de treball, mentre que els centres determinen la distribució geogràfica dels recursos.
Plantejament	Variació independent del nombre de grups (5a: 5 centres fix, $n_g \in \{100, \ldots, 400\}$) i del nombre de centres (5b: 100 grups fix, $n_c \in \{5, \ldots, 35\}$).
Hipòtesi	5a: el temps creixerà superlinealment i la H_{final} creixerà proporcionalment. 5b: la qualitat millorarà a mesura que augmentin els centres (més recursos), i el temps creixerà moderadament perquè el factor de ramificació de SwapGroups no depèn de n_c.
Mètode	HC, H_1, GreedyPro, SG+MG+SO. 10 rèpliques per punt.

Taula 7.13: Plantejament de l'Experiment 5

5a: Augment de grups (5 centres fix)

n_g	H_{final} (mitjana ± std)	Temps (ms)	Factor temps
100	$2\,617.5 \pm 154.5$	53.9	1.0×
150	$3\,896.6 \pm 239.3$	158.4	2.9×
200	$5\,113.3 \pm 283.5$	489.1	9.1×
250	$6\,342.7 \pm 309.5$	1 197.6	22.2×
300	$7\,621.2 \pm 357.2$	3 188.4	59.2×
350	$8\,841.7 \pm 436.1$	5 488.2	101.8×
400	$10\,027.6 \pm 450.0$	10 036.3	186.2×

Taula 7.14: Augment de grups — HC, 5 centres, 10 rèpliques

5b: Augment de centres (100 grups fix)

n_c	H_{final} (mitjana ± std)	Temps (ms)	Reducció H
5	$2\,617.5 \pm 154.5$	40.0	—
10	$2\,356.4 \pm 161.1$	57.1	−10.0%
15	$2\,227.3 \pm 127.4$	101.0	−14.9%
20	$2\,131.1 \pm 132.9$	129.6	−18.6%
25	$2\,038.6 \pm 137.6$	162.4	−22.1%
30	$1\,977.8 \pm 144.8$	164.8	−24.4%
35	$1\,937.0 \pm 127.8$	154.0	−26.0%

Taula 7.15: Augment de centres — HC, 100 grups, 10 rèpliques. Reducció respecte $n_c = 5$.

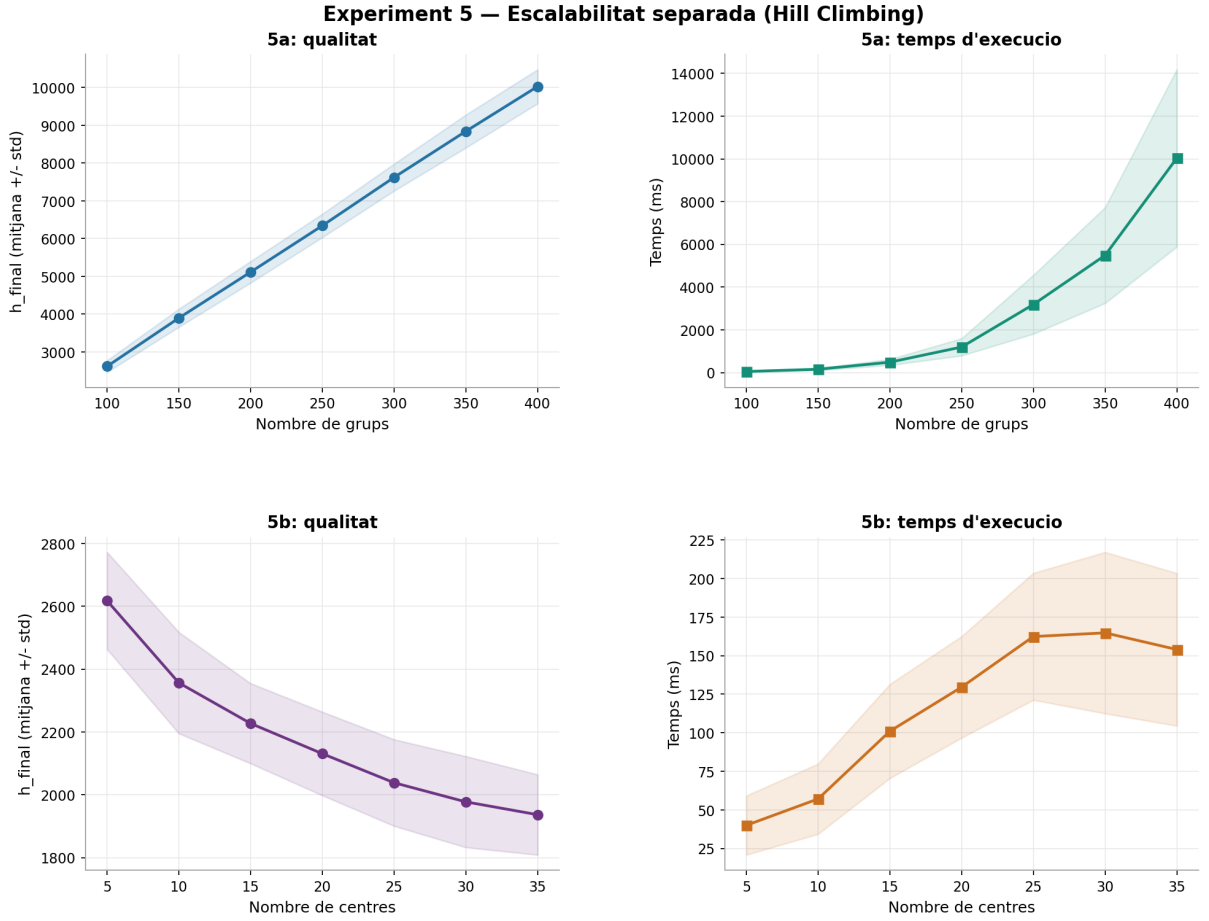

Figura 7.9: Escalabilitat separada: qualitat i temps en funció del nombre de grups amb centres fixes (5a, files superiors) i del nombre de centres amb grups fixos (5b, files inferiors). La banda ombrejada indica ±1 desviació estàndard sobre 10 rèpliques.

Anàlisi i conclusions

5a – Creixement de grups. El temps creix de manera molt superlineal: doblar els grups ($100 \to 200$) multiplica el temps per $\approx 9\times$, i quadruplicar-los ($100 \to 400$) suposa un factor de $186\times$. Això és consistent amb la dominació de SwapGroups ($O(G^2)$ per iteració), que al doblar G quadruplica el cost de generar el veïnat. H_{final} creix de forma gairebé lineal ($\approx +187$ per cada 50 grups afegits), com era d'esperar.

5b – Creixement de centres. El comportament és radicalment diferent: multiplicar per set els centres ($5 \to 35$) redueix H_{final} un 26% (de 2 617.5 a 1 937.0) amb un increment de temps de tan sols $\approx 3.8\times$ (de 40 a 154 ms). Afegir centres distribueix geogràficament els recursos, reduint les distàncies de vol globals. El temps creix moderadament perquè el factor de ramificació de SwapGroups segueix sent $O(G^2)$ independentment de n_c.

7.7 Experiment 6: Efecte del Nombre d'Helicòpters per Centre

Observació	Augmentar el nombre d'helicòpters per centre incrementa la capacitat paral·lela de rescat sense canviar la distribució geogràfica dels recursos.
Plantejament	Variem $H_c \in \{1, 2, 3, 5\}$ mantenint l'escenari base (5 centres, 100 grups).
Hipòtesi	Qualitat: més helicòpters reduirà H_{final}, perquè es poden atendre més grups en paral·lel. Temps: augmentarà perquè el veïnat creix. La millora en qualitat serà decreixent (rendiments marginals).
Mètode	HC, H_1, GreedyPro, SG+MG+SO. 10 rèpliques per configuració.

Taula 7.16: Plantejament de l'Experiment 6

Resultats

H_c (helis/centre)	$H_{\textbf{final}}$ **(mitjana ± std)**	**Temps (ms)**	**Reducció** H
1	$2\,617.5 \pm 154.5$	54.7	—
2	$2\,552.4 \pm 165.7$	82.5	-2.5%
3	$2\,493.0 \pm 157.8$	91.5	-4.8%
5	$2\,384.4 \pm 165.6$	160.3	-9.0%

Taula 7.17: Efecte del nombre d'helicòpters per centre — HC, 5 centres, 100 grups, 10 rèpliques

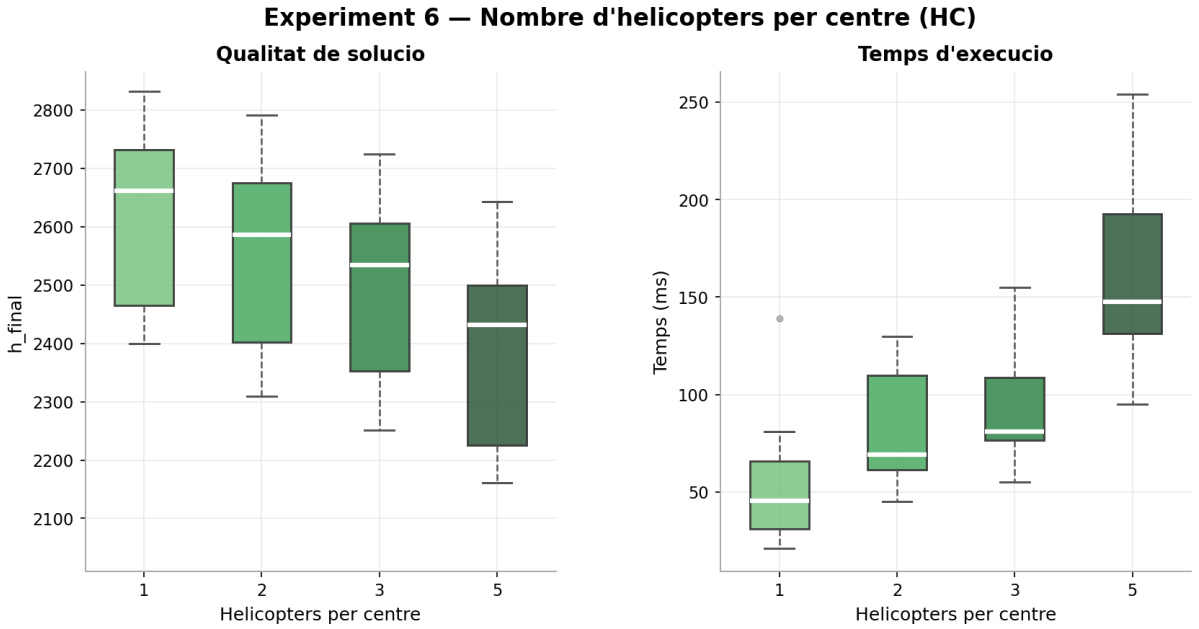

Figura 7.10: Boxplots de H_{final} (esquerra) i temps d'execució (dreta) en funció del nombre d'helicòpters per centre (10 rèpliques). La degradació de verd indica valors creixents de H_c.

Anàlisi i conclusions

Augmentar el nombre d'helicòpters millora la qualitat de forma progressiva: passar d'1 a 5 helis/centre redueix H_{final} un 9.0% (de 2 617.5 a 2 384.4). La millora marginal per helicòpter afegit és decreixent: el salt de 1 a 2 helis aporta 65.1 punts, el de 2 a 3 aporta 59.4 i el de 3 a 5 aporta 108.6 (54.3 per heli). El temps es casi dobla en passar de 1 a 5 helis ($\times$2.9), significativament menys que el creixement de la qualitat.

Comparant amb l'Experiment 5b, on afegir centres (amb 1 heli cadascun) millorava la qualitat un 26% amb un cost temporal de $\times$3.8, es veu que les dues estratègies no són equivalents en eficiència. Amb $H_{\text{total}} = 5 \times 5 = 25$ helis (equivalent a Exp 5b, $n_c = 5$, $H_c = 5$) en l'Experiment 6, la H_{final} és 2 384.4, mentre que amb els 25 helis distribuïts en 5 centres addicionals (Exp 5b $n_c = 10$, $H_c = 2$) la H_{final} és 2 394.4, resultats molt similars en aquest cas particular. La distribució geogràfica és, doncs, tan important com el nombre total de recursos.

7.8 Experiment 7: Heurística H_2 i Ponderació

Observació	H_1 minimitza el temps global sense prioritzar explícitament els ferits. Afegir un terme H_2 que penalitzi el retard dels ferits pot millorar l'atenció urgent al cost d'un lleuger empitjorament global.
Plantejament	Usem $H_{\text{comb}} = H_1 + w \cdot H_2$ amb $w \in \{0, 1, 2, 4, 8, 16\}$. $w = 0$ correspon a H_1 pura i serveix de referència.
Hipòtesi	Amb $w > 0$ la cerca prioritzarà els ferits i reduirà H_2, però incrementarà H_1. La millora de H_2 serà gran per a valors petits de w i s'estabilitzarà a partir d'un llindar. SA aconseguirà millors H_2 que HC per al mateix w gràcies a la seva capacitat d'escapar de mínims locals.
Mètode	Escenari base. HC i SA, GreedyPro, SG+MG+SO. 10 rèpliques per w.

Taula 7.18: Plantejament de l'Experiment 7

Diferència entre H_1 i H_2

H_1 mesura el temps total de tots els helicòpters fins a rescatar tots els grups, ponderant $\times 2$ el temps de recollida dels grups ferits (prioritat 1). H_2 afegeix una penalització proporcional al temps que espera el darrer ferit sense ser rescatat, introduint una asimetria explícita que incentiva la cerca a atendre els ferits aviat, fins i tot si això suposa sacrificar l'eficiència global.

Resultats

w	H_1^{HC}	H_2^{HC}	H_1^{SA}	H_2^{SA}	T_{HC} (ms)
0	2 617.5	15 584.1	2 601.1	15 467.1	50.2
1	2 800.1	10 559.5	2 722.1	7 804.8	311.2
2	2 816.9	9 749.8	2 728.7	7 434.0	364.7
4	2 798.1	9 318.2	2 763.6	7 587.9	451.7
8	2 830.6	9 781.0	2 764.5	7 440.4	422.9
16	2 814.5	9 641.1	2 758.6	7 162.6	364.3

Taula 7.19: Efecte de la ponderació w sobre H_1 i H_2 finals — HC i SA, 10 rèpliques

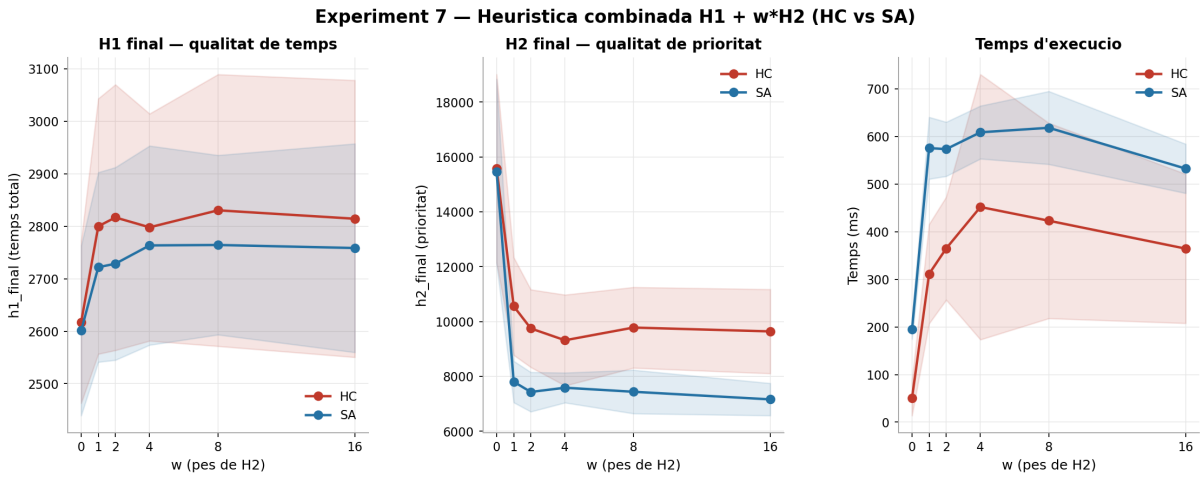

Figura 7.11: Evolució de H_1 (esquerra), H_2 (centre) i temps d'execució (dreta) en funció del pes w, per a HC i SA. La banda ombrejada indica ± 1 desviació estàndard sobre 10 rèpliques.

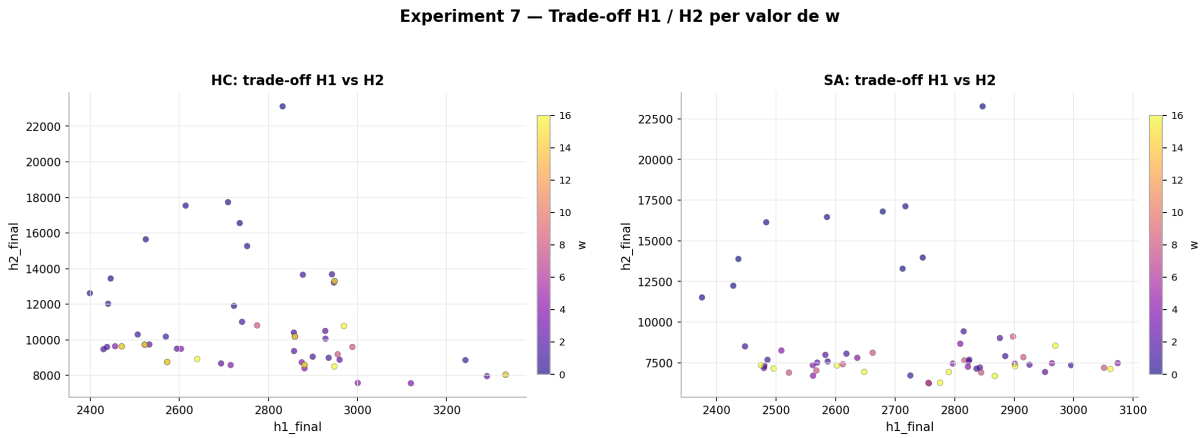

Figura 7.12: Diagrama de dispersió H_1 vs. H_2 per a HC (esquerra) i SA (dreta). Cada punt és una rèplica; el color indica el valor de w (escala plasma, de porpra $w = 0$ a groc $w = 16$). Un bon equilibri es troba a la cantonada inferior esquerra.

Anàlisi i conclusions

Els resultats confirmen la hipòtesi. Ja amb $w = 1$, H_2 es redueix dràsticament: un $\approx 32\%$ per a HC (de 15 584 a 10 560) i un $\approx 50\%$ per a SA (de 15 467 a 7 805). Aquest guany en urgència dels ferits té un cost modest sobre H_1: un $+7.0\%$ per a HC i un $+4.7\%$ per a SA.

A partir de $w = 2$ les millores addicionals de H_2 per a HC es tornen marginals i no monotònes ($9\,750 \to 9\,318 \to 9\,781 \to 9\,641$), indicant que HC queda atrapat en mínims locals on tot moviment que millora H_2 empitjora H_1 i viceversa. SA mostra un comportament més consistent, amb H_2 progressivament decreixent a mesura que w augmenta, i assolint la millor H_2 amb $w = 16$ (7 163).

La gran diferència entre HC i SA en H_2 (per exemple, 9 318 vs. 7 588 a $w = 4$) confirma que escapar de mínims locals és especialment important quan l'heurística té dos objectius

en tensió: SA accepta transitoris que empitjoren H_1 per arribar a zones de l'espai on H_2 és significativament millor.

El millor equilibri entre eficiència global i urgència dels ferits s'assoleix amb $w = 1$–2 per a HC (gran reducció de H_2 amb impacte moderat sobre H_1) i amb $w \geq 8$ per a SA si es prioritza al màxim l'atenció als ferits.

Capítol 8

Conclusions

Conclusions Generals

Aquesta pràctica ha permès dissenyar, implementar i avaluar experimentalment una solució completa al problema de rescat en desastres naturals mitjançant dues metaheurístiques de cerca local: Hill Climbing (HC) i Simulated Annealing (SA). A continuació es resumeixen les principals conclusions extretes de cada aspecte del treball.

Representació i modelatge. La representació dual `(grupoHeli, rutas)` ha demostrat ser la solució adequada per equilibrar eficiència computacional i facilitat d'implementació dels operadors. Gràcies a l'accés en $O(1)$ a l'helicòpter d'un grup i al càlcul heurístic en $O(G)$, la cerca local pot generar i avaluar milers de veïns per segon fins i tot en instàncies de centenars de grups. La redundància controlada entre les dues estructures és una decisió de disseny justificada i beneficiosa.

Solució inicial. L'estratègia d'inicialització té un impacte significatiu tant en la qualitat final com en el temps d'execució. El generador `StartGreedyPro`, basat en inserció de mínim cost, produeix solucions inicials molt properes a l'òptim local, reduint el temps d'execució de HC en gairebé $5\times$ respecte al generador aleatori i assolint una H_{final} estadísticament millor ($p < 0.0001$). Sorprenentment, el generador `StartGreedy` simple no difereix significativament del generador aleatori en qualitat final ($p = 0.146$), cosa que indica que HC és capaç de recuperar-se d'una mala inicialització, però a un cost temporal elevat.

Operadors. Els tres operadors implementats (`SwapGroups`, `MoveGroup`, `SwapOrder`) formen un conjunt complet i eficient. `SwapGroups` és el motor principal de millora (reducció del $\sim 40\%$ sobre H_{inicial}) i domina el factor de ramificació ($\sim 91\%$ del total). La combinació `SG+MG+SO` obté la millor qualitat en tots els escenaris provats, i l'avantatge de `MoveGroup` s'accentua quan hi ha múltiples helicòpters per centre, validant la hipòtesi que aquest operador és especialment útil en escenaris amb recursos paral·lels.

Funcions heurístiques. H_1 (temps total) és una funció d'eficiència global senzilla i consistent. H_2 (temps total amb penalització de prioritat) permet incorporar l'urgència mèdica dels ferits de forma explícita. Amb un pes $w = 1$, H_2 redueix el temps màxim

fins al darrer ferit rescatat en un 32% (HC) i un 50% (SA) amb un cost addicional sobre H_1 de tan sols un 7% i 4.7% respectivament. A partir de $w = 4$, les millores addicionals sobre H_2 es tornen marginals per a HC, indicant que HC queda atrapat en mínims locals on els dos objectius entren en conflicte.

Comparació HC vs. SA. HC és superior en qualitat per a instàncies petites i mitjanes amb el temps disponible; SA és preferible quan el temps d'execució és crític o quan el problema té dos objectius en tensió (com H_1 vs. H_2). Per a l'escenari base ($G = 100$, $C = 5$), el SA ben calibrat (k = 10, $\lambda = 0.001$, *steps* = 150 000) supera HC en un 0.5% en H_1 i en un $\sim 20\%$ en H_2. No obstant, quan el problema escala (a partir de ~ 15 centres / 300 grups), HC torna a ser millor en qualitat perquè SA explora una fracció decreixent de l'espai amb el nombre de passos fixat. El temps d'HC creix $\sim 1235\times$ en multiplicar per $5\times$ el tamany del problema, mentre que el de SA ho fa de forma gairebé lineal.

Escalabilitat. Augmentar el nombre de grups és el factor que més penalitza HC ($O(G^2)$ per iteració degut a `SwapGroups`), mentre que augmentar el nombre de centres redueix la qualitat de la solució (-26% en H_{final} en multiplicar per $7\times$ els centres) amb un cost temporal molt moderat ($\times 3.8$). Distribuir geogràficament els recursos és tan efectiu com augmentar el nombre total d'helicòpters en un mateix centre: ambdues estratègies produeixen resultats similars quan el nombre total d'helicòpters és equivalent.

En resum, la cerca local és una estratègia molt adequada per a aquest problema NP-difícil. La combinació `StartGreedyPro + SG+MG+SO` amb HC és la millor opció quan la qualitat és prioritària i el problema és de mida petita o mitjana. Per a problemes grans o quan cal prioritzar els ferits, SA amb $w \geq 1$ ofereix un millor equilibri entre qualitat, urgència i temps d'execució.

Línies de Treball pel Futur

Tot i els bons resultats obtinguts, diverses extensions podrien millorar la solució:

- **Operadors més expressius:** L'operador `SwapOrder` actual només intercanvia grups adjacents dins la mateixa ruta. Implementar un operador de tipus *2-opt* (intercanvi de dos segments no adjacents) o *Or-opt* (desplaçament de blocs de grups consecutius) podria millorar significativament l'ordre de visita dins de cada helicòpter, especialment en rutes llargues.
- **Ajust Dinàmic de Paràmetres:** La temperatura inicial i el schedule de refredament del SA s'han ajustat de forma estàtica mitjançant experiments. Un esquema adaptatiu que modifiqui λ i k en funció de la taxa d'acceptació observada durant la cerca podria millorar la convergència i reduir la necessitat d'una exploració manual exhaustiva dels paràmetres.
- **Reinicialitzacions Múltiples (Iterated Local Search):** Tant HC com SA convergen a un únic òptim local per execució. Una estratègia d'*Iterated Local Search* que pertorbi la solució actual en arribar a un òptim local (per exemple, reubicant aleatòriament un 10% dels grups) permetria explorar múltiples conques d'atracció i obtenir solucions de major qualitat en el mateix temps total.

- **Paral·lelització:** Atès l'elevat cost computacional de HC en escenaris de gran escala (creixement $O(G^2)$ per iteració), una implementació paral·lela de la generació i avaluació del veïnat, o l'adopció d'una estratègia *first improvement* en lloc de *best improvement*, podria reduir significativament el temps d'execució tot mantenint la qualitat de les solucions.
- **Restriccions del Problema Real:** El model actual assumeix que tots els helicòpters estan sempre disponibles i que les condicions de vol són ideals. En un escenari real caldria considerar averies, restriccions de combustible, condicions meteorològiques adverses i zones d'accés restringit. Incorporar-les com a restriccions toves a la funció heurística, penalitzant les solucions que les violen, seria una extensió natural i de gran rellevància pràctica.

En definitiva, tot i que els mètodes actuals ofereixen solucions de bona qualitat en entorns de cerca reduïts, la recerca futura s'hauria de centrar en adaptar els algorismes a escenaris més complexos, millorant la robustesa, l'eficiència i la capacitat d'adaptació davant de condicions canviants i múltiples criteris d'optimització simultanis.

Apèndix A

Treball en Equip

A.1 Distribució de tasques

Taula A.1: Distribució de tasques entre els membres del grup

Tasca	**Responsable**	**Col·laboradors**
Implementació RescueState	Eloi	–
Implementació operadors	Eloi	Guillem
Estratègies inicialització	Eloi	–
Funcions heurístiques	Guillem	Eloi
Experiments i anàlisi	Eloi	–
Documentació	Eloi	Guillem

Apèndix B

Treball d'innovació

B.1 Ollama

Ollama és una eina avançada de codi obert dissenyada per a l'execució local de models de llenguatge extensos (LLM) basada en intel·ligència artificial i aprenentatge profund. Està dissenyat per augmentar la privadesa i l'accessibilitat, permetent funcions com la generació de text, el resum de documents i l'assistència en programació directament des de l'ordinador de l'usuari sense dependre del núvol.

Utilitza arquitectures de xarxes neuronals optimitzades i tecnologies de quantificació per interpretar i generar llenguatge natural en temps real utilitzant el maquinari local (GPU i CPU). La seva evolució cap a la democratització de la IA ha transformat la indústria del programari, millorant la seguretat de les dades i avançant cap a un futur amb una IA més oberta, privada i eficient.

Aspectes	**Encarregat**
Descripció del producte o servei i de la seva innovació	Eloi Mas
Descripció de les tècniques d'IA que s'han utilitzat	Guillem Revuelta
Descripció de com han estat utilitzades les tècniques	Guillem Revuelta
Explicació de per què és un producte/servei innovador	Eloi Mas
Impacte de la innovació feta al producte en l'empresa	Guillem Revuelta
Impacte de la innovació feta al producte en l'usuari o en la societat	Eloi Mas

Referències

- 24-2 `https://ollama.com/library` (Ollama Model Library)
- 24-2 `https://ollama.com/blog` (Ollama AI and Updates)
- 24-2 `https://github.com/ollama/ollama` (Ollama Open Source Repository)

- 25-02 `https://ollama.com/download` (Ollama Hardware Requirements)
- 25-02 `https://en.wikipedia.org/wiki/Large_language_model` (Informació general LLM)
- 25-02 `https://shorturl.at/local-ai-guide` (Guia d'IA Local i Privadesa)